APEXMATCH

Distributed High-Throughput Matching Engine & Asset Ledger

System Requirements Specification (SRS) & Implementation Blueprint

Document Code: SRS-APEXMATCH-Distributed

Target Role: Backend / Systems Engineer

System Tier: Mid-to-Senior Level Portfolio Project

Core Stack: Node.js, Redis, RabbitMQ, MySQL, MongoDB

1. System Objective & Core Concept

The objective of this project is to construct a production-ready, distributed backend capable of managing high-frequency concurrent matching operations. Instead of building a standard application that directly modifies databases upon client interaction, this architecture decouples the immediate **In-Memory Matching Logic (The Hot Path)** from the permanent **Database Persistence Layer (The Cold Path)**.

This blueprint is explicitly generalized. It focuses purely on the computer science problem of **atomic inventory allocation and balance synchronization under heavy concurrency**. You can skin this project to match any business domain that involves a high volume of instant reservations, purchases, or trades.

2. Domain Selection & Customization Matrix

Before writing code, choose one of the business domains below. The architecture under the hood remains completely identical regardless of your choice; only your variable naming and data models change.

Option A: Flash Ticket Booking

Example Scenario: Concert/IPL ticket sales where 50,000 users grab 5,000 premium seats instantly.

- **The Pool:** Available seat numbers per section.
- **The Buyers:** Users offering to buy seats.
- **The Sellers:** The event organizer releasing inventory.
- **Match Definition:** Buyer payment successfully pairs with an unassigned seat token.

Option B: Financial Asset / Crypto Exchange

Example Scenario: High-speed trading platform matching buy and sell intent at microsecond variations.

- **The Pool:** Stock shares or digital currency assets.
- **The Buyers:** Users posting high-bid buying prices.
- **The Sellers:** Users posting low-ask selling prices.
- **Match Definition:** Buyer bid price meets or exceeds seller ask price.

Option C: On-Demand Ride/Delivery Surge

Example Scenario: High-density urban centers matching active orders to available fulfillment entities.

- **The Pool:** Drivers or delivery personnel.
- **The Buyers:** Commuters/Customers ordering trips.
- **The Sellers:** Independent service operators accepting fares.
- **Match Definition:** User budget coordinates directly with nearby provider availability thresholds.

Option D: Limited-Drop E-Commerce Marketplace

Example Scenario: Exclusive sneakers or product releases selling out within seconds.

- **The Pool:** Verified localized warehouse inventory counts.
- **The Buyers:** Eager customers checked in at checkout queues.
- **The Sellers:** Merchants releasing strict stock allocations.
- **Match Definition:** Reservation request locks available item ID before exhaustion.

3. The Architecture: Step-by-Step Data Flow

The system handles data through two primary processing speed tracks to maximize processing speed while maintaining financial integrity:

3.1 Phase 1: Ingress & Validation (0 - 3 Milliseconds)

- The client initiates a persistent connection via **WebSockets** or launches a high-speed HTTP POST request containing an order payload.

- The Ingress Gateway validates the structural formatting of the data and verifies the authentication token (JWT).
- **Idempotency Check:** The gateway evaluates an incoming unique string key (`idempotencyKey`) sent by the client. It stores this key temporarily in Redis for 5 seconds. If a duplicate key arrives due to network retries, it is instantly rejected to prevent duplicate transactions.

3.2 Phase 2: Structural Verification & Ledger Freezing (3 - 10 Milliseconds)

- The Gateway verifies account balances inside the relational database. It checks if the customer possesses enough capital or inventory.
- To eliminate race conditions, the application executes a **Row-Level Lock** in MySQL using the explicit command syntax: `SELECT * FROM user_wallets WHERE user_id = X FOR UPDATE;`
- The system subtracts the required capital from the user's **Available Balance** and adds it to a **Frozen Balance** state. The database transaction commits immediately. This safeguards the money so it cannot be double-spent across multiple simultaneous server instances.

3.3 Phase 3: In-Memory Engine Execution (10 - 12 Milliseconds)

- The Gateway transfers the verified order directly into the **In-Memory Matching Engine** layer.
- The Engine implements **Redis Sorted Sets (ZSET)** to index active, unmatched demands. Buy orders are indexed from highest price/priority to lowest, while sell orders are sorted from lowest price/priority to highest.
- The engine compares the top-ranked buyers against the top-ranked sellers in memory. If a match occurs, the engine deletes the matched items from the Redis `ZSET`. If no match occurs, the request rests in the sorted set until a corresponding counter-party arrives or a cancellation command is fired.

3.4 Phase 4: Async Decoupling & Permanent Settlement (12 Milliseconds onwards)

- Upon establishing a match, the engine broadcasts an instantaneous notification payload backward through open WebSockets to update the user interfaces under 15 milliseconds.
- Concurrently, the engine creates a permanent processing payload and throws it into a high-capacity message broker: **RabbitMQ**. This acts as our safety cushion, separating our memory calculations from slow disk modifications.
- Independent, background **Worker Services** consume the events from RabbitMQ:
  - **Ledger Worker:** Opens an atomic transaction inside **MySQL**, releases the frozen balances, moves the capital permanently to the seller, transfers ownership of the asset/ticket, and logs a finalized entry.
  - **Audit Worker:** Appends the full unstructured event chain directly into a **MongoDB Time-Series Collection** for compliance tracking, history preservation, and error resolution.

4. Core Technical Requirements & Guardrails

- **No Direct DB Writes on Core Flow:** The Hot Path must never block execution to wait for a database query disk write. All active matching logic must take place in memory.

- **Graceful Cancellation Capability:** If an item remains unmatched, the user can emit a cancel packet. The engine executes a low-overhead `ZREM` command in Redis to remove the record. It publishes an `ORDER_CANCELLED` message to RabbitMQ, causing the Ledger Worker to transfer frozen capital safely back to the user's available balance in MySQL.
- **Strict Fault Tolerance:** If the matching engine node crashes mid-operation, the system state must be fully rebuildable. When the service boots back up, it must scan the active ledger state from MySQL and re-populate the Redis memory workspace cleanly before accepting new customer requests.

5. Beginner-Friendly 8-Week Implementation Timeline

Treat this project like a professional product shipment. Dedicate 6-10 hours a week across this structured 8-week developmental plan:

Timeline	Engineering Milestone Focus	Actionable Implementation Deliverables
Week 1-2	Database Prototyping & Row-Locking Ledger	Install MySQL and MongoDB. Write localized schema scripts for wallets, items, and transactions. Code a raw Node.js script that opens a balance verification query using row-level locking (`FOR UPDATE`) to test race condition blocks.
Week 3	Redis Order Book Design	Set up Redis locally. Write Node.js code utilizing the `ioredis` library to push, pop, and read records from a Redis Sorted Set (`ZSET`) based on priority tracking parameters.
Week 4	Core Matching Algorithm Engine	Write the absolute pure JavaScript matching logic script. It must check the highest buyer against the lowest seller inside Redis, evaluate match parameters, and delete records cleanly when conditions are satisfied.
Week 5	RabbitMQ Async Integration	Install RabbitMQ. Write a background Worker script that safely connects to a queue, catches event payloads generated from the Week 4 matching loop, and executes permanent updates inside MySQL and MongoDB.
Week 6	Gateway API & WebSocket Layer	Build an Express.js server wrapper acting as the API Ingress. Set up authentication checks and establish a native WebSocket server connection pipeline (`ws` package) to handle high-frequency incoming calls.
Week 7	Docker Practicalization & Composability	Write individual `Dockerfiles` for the Node.js API and workers. Construct a single, structured `docker-compose.yml` linking Node, Redis, RabbitMQ, MySQL, and MongoDB via a custom secure network. Verified by executing `docker-compose up` .
Week 8	High-Load Testing & Optimization	Install a professional load generation utility like **k6** or **Autocannon**. Bombard your gateway with 2,000 concurrent requests per second. Fine-tune your database indices and check memory usage to guarantee stability.

6. The Public Learning Blueprint: Job Hunting Guide

To break through industry filters, you must build in public. Recruiters do not look for perfection; they look for problem-solving transparency. Use this strategy to share your journey:

6.1 Weekly LinkedIn Posting Calendar & Topic Inspiration

- **Week 2 Post (The Concurrency Problem):** Explain what a race condition is. Write a text post: *"Most junior developers think building a payment wallet is easy. This week, I learned that if a user double-clicks a buy button, they can spend money they don't have. Here is how I used MySQL row-level locking (* `FOR UPDATE` *) to block parallel transactions safely."* Attach a screenshot of your terminal console blocking a simulated parallel attack.
- **Week 4 Post (Data Structures Choice):** Share why you chose Redis. *"To handle thousands of operations per second, disk storage is too slow. This week, I designed an in-memory queue workspace using Redis Sorted Sets. It automatically ranks requests by priority variations instantly in memory, removing overhead."*
- **Week 6 Post (Asynchronous Architecture):** Write about the introduction of message brokers. *"What happens if your database slows down while users are clicking buttons? Your whole backend crashes. This week, I decoupled my application using RabbitMQ. The matching engine does the work instantly and tosses a message to a queue. The database processes it smoothly in the background."*
- **Week 7 Post (The Docker Evolution):** Celebrate containerization. *"No more 'it works on my machine' problems. Today, I containerized my entire 5-tier microservices architecture using Docker Compose. With one command, I can spin up a fully isolated environment anywhere."* Show a 10-second video of your terminal running `docker-compose up`.

6.2 Technical Video Recording Guide (The Final Showpiece)

At the end of Week 8, record a 3-minute video using **Loom** or **OBS Studio**. Pin this video directly to the top of your LinkedIn profile and place the link inside your resume contact section.

1. **0:00 - 0:30 (The Hook):** Do not start with your name or degree. Open the video showing your terminal workspace. Say: *"This is ApexMatch. It is a distributed microservices backend built in Node.js that handles 2,000 concurrent operations per second without dropping a single record or double-allocating balances."*
2. **0:30 - 1:30 (The System Architectural Walkthrough):** Open your system diagram. Walk through how the Hot Path (Redis) is split apart from the Cold Path (MySQL/MongoDB via RabbitMQ) to maintain low-latency response profiles.
3. **1:30 - 2:30 (The Practical Execution Load Test):** Fire up your **k6 / Autocannon** scripting platform on camera. Show the server console handling thousands of requests hitting the system. Point to the screen and say: *"Notice how the HTTP layer continues accepting actions at millisecond latency while RabbitMQ holds the transaction load safely to prevent database throttling."*
4. **2:30 - 3:00 (The Conclusion):** Summarize what you learned about system architecture, error handling, and production-ready containerization. Invite viewers to check out your comprehensive GitHub repository.